

Master's Thesis

Adaptive Rounding for Efficient Post-Training Quantization of Deep Neural Networks

48246347 Piye Liu

Supervisor Prof. Hiroshi Nakamura

August, 2026

Department of Information Physics and Computing
Graduate School of Information Science and Technology
The University of Tokyo

概要

Abstract in Japanese should be written here.

Abstract

300 words abstract in English should be written here.

Contents

Chapter 1	Introduction	1
1.1	Background	1
1.2	Problem Definition	1
1.3	Contributions	1
Chapter 2	Preliminary	2
2.1	Compressibility of Deep Neural Networks	2
2.2	Small Language Models	4
2.3	Baseline Round-to-Nearest Quantization	6
2.4	Quantization is Weight Perturbation	6
2.5	Effects of Perturbation of Model Weights	7
Chapter 3	Literature Review	12
3.1	AdaRound	12
3.2	Inversion Method	14
Chapter 4	Proposed Method	15
4.1	AdaRound with Gradient Awareness	15
Chapter 5	Evaluation	17
Chapter 6	Conclusion	18
	Bibliography	19
A	Appendix	20

Chapter 1

Introduction

1.1 Background

1.2 Problem Definition

1.3 Contributions

Our contributions in this work are twofold: First, we demonstrate a novel inversion method against AdaRound to recover coarse outlines that represent its calibration data, motivating the need to consider privacy in data-dependent quantization method. Second, we propose a gradient-aware adaptive rounding method based on AdaRound, and empirically validate the proposed method on two language models.

Chapter 2

Preliminary

2.1 Compressibility of Deep Neural Networks

In this section, we describe a simple way to quantify the compressibility of models. The source coding theorem [1] states that:

- n i.i.d. (independent and identically distributed) random variables, each with entropy $H(X)$, can be compressed to $nH(X)$ bits on average, with negligible information loss, as $n \rightarrow \infty$
- Conversely, if they are compressed to fewer bits on average, loss of information must occur.

In this case, $H(X)$, the entropy of a random variable, is the expected amount of information needed to describe it, and can be computed by the following.

$$H(X) := - \sum_{x \in X} p(x) \log_2 p(x) \quad (2.1)$$

Note that the log base determines the units: $\log_2$ yields bits, $\log_{10}$ yields dits, and $\ln$ yields nats.

Consider a model with weights W , each stored in b bits, and there are n weights in total. The total number of bits occupied by the model is therefore $n \cdot b$. This is the capacity of the model: it is the maximum amount of information the weights could carry if every possible bit pattern were equally likely and the weights were statistically independent.

In practice, however, models rarely use their full capacity. We can measure this by treating its weights as independent draws from a categorical distribution over these possible values.

$$\mathcal{X} = \{0, 1, \dots, 2^b - 1\} \quad (2.2)$$

We can use Maximum Likelihood Estimation (MLE) to estimate the distribution as follows.

$$\hat{p}(x) = \frac{1}{n} \sum_{k=1}^n \mathbf{1}(W_k = x), \text{ where } \mathbf{1}(p) = \begin{cases} 1 & \text{if } p \\ 0 & \text{if } \neg p \end{cases} \quad (2.3)$$

And we can estimate the entropy per weight as follows.

$$\hat{H}(W) := - \sum_{x \in \mathcal{X}} \hat{p}(x) \log_2 \hat{p}(x) \quad (2.4)$$

Note that if a certain value does not appear at all in the model weights, there will be an undefined term $0 \ln 0$ in the above calculation. We use a convention in modern information

theory to mitigate this, by using an extension of $x \ln x$ in the form of the following [2, eq. 1.1.2].

$$f(x) = \begin{cases} 0, & x = 0 \\ x \ln x, & x > 0 \end{cases} \quad (2.5)$$

This allows us to treat $0 \ln 0$ as 0. We can see that the total information stored in the weights would be approximately $n \cdot \hat{H}(W)$ bits. But the maximum possible total entropy of the model is $n \cdot b$ bits (achieved by the uniform distribution), it therefore follows that the capacity utilization by a model is the following.

$$U(W) := \frac{\hat{H}(W)}{b} \quad (2.6)$$

Note that $U \in [0, 1]$, and $U = 1$ implies the model is nearly incompressible losslessly, while $U < 1$ implies the model can be losslessly compressed to $1 - U$ times its current size. For example, if $U = 0.5$, the model can be compressed to half its size with negligible information loss.

Qwen 3 [3] is a state-of-the-art language model in various sizes. Now that we have a method to quantify the compressibility of models, let us evaluate the compressibility of the 4B model, which is provided in three data types, of widths 16, 8, and 4. We report the results in Table 2.1.

Table 2.1: Entropy Estimate of Qwen 3 4B Weights

Data Type	$\hat{H}(W)$ in bits	$U(W)$
bfloat16	10.58	$10.58/16 \approx 66\%$
fp8	6.63	$6.63/8 \approx 82.9\%$
int4	3.67	$3.67/4 \approx 91.8\%$

Consider reproducing with more models

We observe that the bfloat16 model, which is ~ 8 GB on disk, only uses $\sim \frac{2}{3}$ of its capacity. This implies that the model can be compressed to ~ 5.33 GB with negligible information loss. This degree of compressibility motivates us to perform an investigation of the distribution of the weights by plotting their possible values against the probabilities of each value, which we report in Figures 2.1, 2.2, and 2.3.

No axis labels (need to rerun the code to add them)

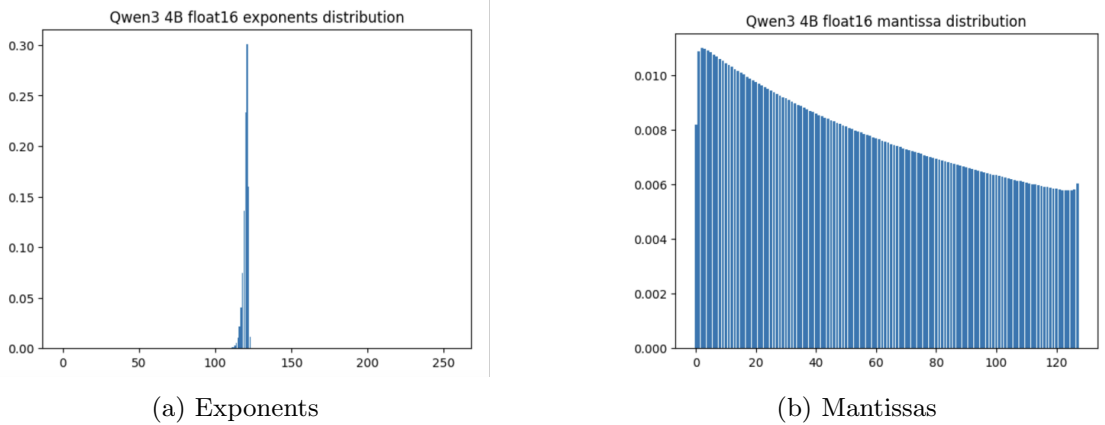


Fig. 2.1: Distribution of Qwen3-4B Weights (bfloat16)

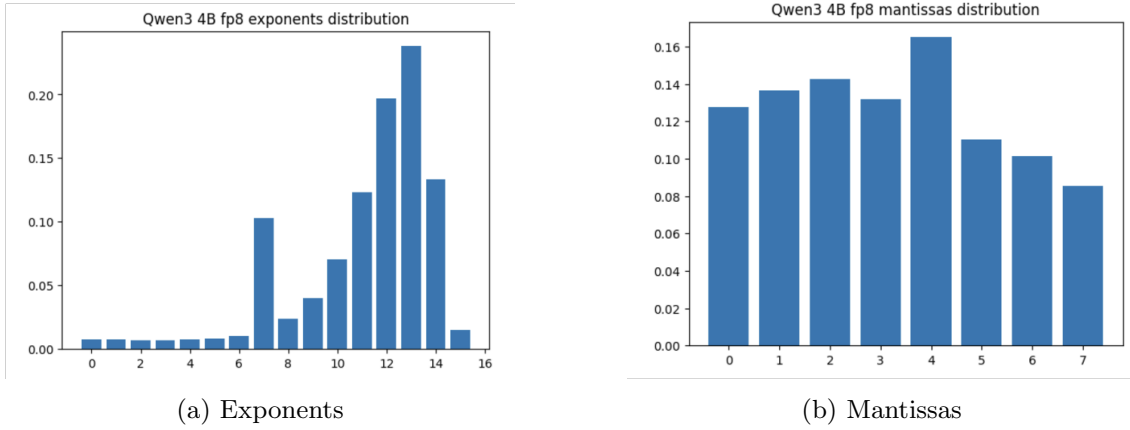


Fig. 2.2: Distribution of Qwen3-4B Weights (fp8)

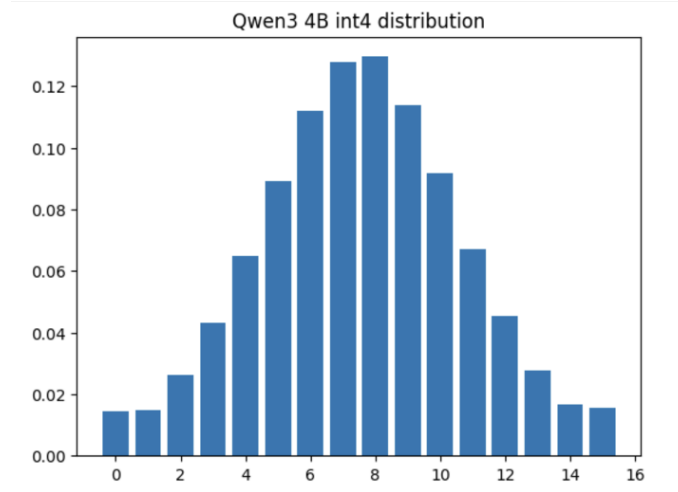


Fig. 2.3: Distribution of Qwen3-4B Weights (int4)

We notice that the full dynamic range of bfloat16 is actually not used. Especially in bfloat16, the exponents are clustered towards the center, with an average of 120 (this means most weights are $m \cdot 2^{-7}$). Several factors may contribute to this phenomenon: weight decay, regularization, and the optimizer (for example, stochastic gradient descent provides implicit optimization [4]).

We therefore conclude that large language models often have much room for compression, and is not using all of its potential capacity. This provides motivation for research in compression methods.

2.2 Small Language Models

In this section, we will examine TinyStories [5], a work that pushes the boundaries of small language models, and provide a few custom language models for further experimentation. By using a low-variance synthetic dataset (children’s stories) generated by GPT-3.5 and GPT-4 containing only words a typical 3-4 year old would understand, they were able to generate coherent output using models with as few as 1 million parameters.

One limitation we observed in this work is their usage of the GPT-Neo [6] tokenizer,

restricted to the top 10,000 tokens. Using a generic tokenizer for a low-vocabulary dataset like TinyStories might result in wasted weights (tokens that never appear in TinyStories), and might not provide the optimal tokenization for our specialized dataset.

As a preliminary experiment, we train two small language models on TinyStories v1 and TinyStories v2, respectively. We use a SentencePiece Byte-Pair Encoding (BPE) tokenizer [7] with a vocabulary size of 1024, and we used a LLaMa-based [8] model with roughly half the parameters of the smallest reference model.

2.2.1 Evaluation: Quantitative Metric

We use perplexity [9] as a quantitative metric to evaluate a language model’s accuracy. For a model θ , a text x of length N , it is defined as the following.

$$\text{Perplexity}(\theta, x) = \left(\prod_{i=1}^N \frac{1}{p_{\theta}(x_i \mid x_1, x_2, \dots, x_{i-1})} \right)^{\frac{1}{N}} \quad (2.7)$$

In other words, it is the geometric mean of the inverse probabilities of a given text. To show this with an example, suppose we have a model θ and a ground truth text “Mary had a little lamb”. Suppose that the model’s predicted probabilities of this text is the following.

Table 2.2: Example Model Output Probabilities

Item	Value
$p_{\theta}(\text{“Mary”})$	0.5
$p_{\theta}(\text{“had”} \mid \text{“Mary”})$	0.7
$p_{\theta}(\text{“a”} \mid \text{“Mary”, “had”})$	0.9
$p_{\theta}(\text{“little”} \mid \text{“Mary”, “had”, “a”})$	0.6
$p_{\theta}(\text{“lamb”} \mid \text{“Mary”, “had”, “a”, “little”})$	0.9

Then the perplexity of the model on the text is the following.

$$\text{Perplexity} = \left(\frac{1}{0.5} \cdot \frac{1}{0.7} \cdot \frac{1}{0.9} \cdot \frac{1}{0.6} \cdot \frac{1}{0.9} \right)^{\frac{1}{5}} \quad (2.8)$$

$$\approx 1.49 \quad (2.9)$$

Intuitively, perplexity is a measure of how surprised or confused a model is when seeing a certain text. We compute the model’s perplexity over its validation data, then use the result as a metric to evaluate the model’s accuracy (lower is better).

2.2.2 Evaluation: Results and Discussion

We report the validation perplexity of our two models. Note that the original paper only provided models trained on TinyStories v1.

Table 2.3: Evaluation of Our Language Models

	Reference	Ours (TinyStories v1)	Ours (TinyStories v2)
Architecture	GPT-Neo	LLaMa	LLaMa
Parameter Count	1.17M	0.57M	0.46M
Training Data	TinyStories v1		TinyStories v2
Tokenizer (Vocab Size)	GPT-Neo	SentencePiece	SentencePiece
Vocab Size	10000	1024	1024
Perplexity (TinyStories v1)	19.3919	11.9006	—
Perplexity (TinyStories v2)	—	—	8.0994

Add qualitative results (actual model outputs) here

Consider an ablation study to train the same GPT-Neo model, with only a different tokenizer

We observe that our model results in better perplexity than the reference model despite being much smaller. In `float32`, each weight is 4 bytes, so our model is expected to take only $\sim 2.17\text{MiB}$ of disk space (the actual disk space, including the model’s configuration, is $\sim 2.22\text{MiB}$). Models of this scale can be suitable for devices with extremely low compute and/or memory.

We will use our models as a base to conduct several more experiments.

2.3 Baseline Round-to-Nearest Quantization

The baseline for post-training quantization is symmetric uniform quantization using round-to-nearest (RTN) rounding [10]. A model can be quantized to b bits using Algorithm 1, then dequantized using Algorithm 2.

Algorithm 1 Baseline RTN Quantization

Require: W (unquantized weights), b (target bit width)

Ensure: Quantized model W_q

- 1: $q_{\min} \leftarrow -2^{b-1}$
 - 2: $q_{\max} \leftarrow 2^{b-1} - 1$
 - 3: $\text{scale} \leftarrow \frac{\max(W)}{q_{\max}}$
 - 4: $W_q \leftarrow \text{clamp}(\lceil \frac{W}{\text{scale}} \rceil, q_{\min}, q_{\max})$
 - 5: **return** W_q, scale
-

Algorithm 2 Baseline RTN Dequantization

Require: W_q (quantized weights), scale

Ensure: Dequantized weights $\hat{W}$

- 1: $\hat{W} \leftarrow W_q \cdot \text{scale}$
 - 2: **return** $\hat{W}$
-

2.4 Quantization is Weight Perturbation

Let us establish the observation that quantization results in a perturbed model, using an

example. Consider a model θ with parameters $W = \begin{bmatrix} 1.11 \\ 2.22 \\ 3.33 \end{bmatrix}$. By applying the baseline

round-to-nearest quantization from Section 2.3, we obtain the following quantized model.

$$\text{scale} \approx 0.0262, W_q = \begin{bmatrix} 0\text{b}00101010 \\ 0\text{b}01010101 \\ 0\text{b}01111111 \end{bmatrix} = \begin{bmatrix} 42 \\ 85 \\ 127 \end{bmatrix} \quad (2.10)$$

After dequantization, we obtain the following model.

$$W_{\text{reconstructed}} = \text{scale} \cdot W_{\text{quantized}} \approx \begin{bmatrix} 1.1013 \\ 2.2287 \\ 3.33 \end{bmatrix} \quad (2.11)$$

We can see that the model is perturbed by the following.

$$\Delta W = \begin{bmatrix} 1.1013 \\ 2.2287 \\ 3.33 \end{bmatrix} - \begin{bmatrix} 1.11 \\ 2.22 \\ 3.33 \end{bmatrix} = \begin{bmatrix} -0.0087 \\ 0.0087 \\ 0 \end{bmatrix} \quad (2.12)$$

It is important to note the direction of perturbation. In this case, our model moved in the direction of $\begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}$.

2.5 Effects of Perturbation of Model Weights

As we established in Section 2.4, quantization perturbs weights in a certain direction. In this section, we examine the effects of weight perturbation using our TinyStories v2 model, by viewing its loss landscape.

2.5.1 Language Model Optimization Process

To understand our model’s loss landscape, we must first understand its loss function. The goal in optimizing language models is to minimize the difference between the ground truth and the model’s predictions. We measure this using the Kullback-Leibler (KL) divergence [11], defined as follows.

$$D_{KL}(P \parallel Q) = \sum_{x \in X} p_P(x) \ln \frac{p_P(x)}{p_Q(x)} \quad (2.13)$$

The KL-divergence measures how an approximating distribution Q is different from a distribution P . We use one-hot encoding to encode the ground truth into the distribution P , and use the model’s outputs as the approximating distribution Q .

Note that we once again need to employ the convention in Equation 2.5. Let us show why with an example: if we are predicting a word “Hello”, the ground truth P distribution and the predicted Q distribution might be:

Table 2.4: Example of KL-divergence

$\hat{y}$	$p_P(\hat{y})$	$p_Q(\hat{y})$
“Today”	0	0.07
“Hello”	1	0.81
“Cat”	0	0.10
“Rain”	0	0.02

In this case, the KL-divergence is:

$$D_{KL}(P \parallel Q) = \underbrace{0 \cdot \ln \frac{0}{0.07}}_{\text{"Today"}} + \underbrace{1 \cdot \ln \frac{1}{0.81}}_{\text{"Hello"}} + \underbrace{0 \cdot \ln \frac{0}{0.10}}_{\text{"Cat"}} + \underbrace{0 \cdot \ln \frac{0}{0.02}}_{\text{"Rain"}}, \quad (2.14)$$

which contains three undefined terms in the form of $0 \cdot \ln(\frac{0}{n})$. Using the convention from Equation 2.5, we can compute the undefined terms as follows.

$$0 \cdot \ln \frac{0}{n} = \underbrace{0 \ln 0}_{0 \text{ by convention}} - 0 \ln n = 0 \quad (2.15)$$

We then obtain the following.

$$D_{KL}(P \parallel Q) = -\ln(0.81) \approx 0.21 \quad (2.16)$$

When minimizing the KL-divergence between the model predictions and the ground truth, we can use the following observation to ignore the constant $H(P)$ term, or the entropy of the ground truth distribution P .

$$D_{KL}(P \parallel Q) = \underbrace{\sum_{x \in X} p_P(x) \ln p_P(x)}_{-H(P)} - \sum_{x \in X} p_P(x) \ln p_Q(x) \quad (2.17)$$

$$\propto - \sum_{x \in X} p_P(x) \ln p_Q(x) \quad (2.18)$$

When optimizing, we only need to minimize $-\sum_{x \in X} p(x) \ln q(x)$. This is also known as the cross entropy $H(P, Q)$ between distributions P and Q .

The cross entropy for a language model θ on a single token, where P_i is the one-hot encoded ground truth and Q_i is the model's predicted distribution, is the following.

$$H(P_i, Q_i) = - \sum_{x \in X} p_{P_i}(x) \ln p_{Q_i}(x) = -\ln p_\theta(x_i \mid x_{<i}) \quad (2.19)$$

The loss function, the average cross-entropy per token over a sequence of length N , is the following.

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N H(P_i, Q_i) = -\frac{1}{N} \sum_{i=1}^N \ln p_\theta(x_i \mid x_{<i}) \quad (2.20)$$

Note that this is exactly the natural logarithm of the perplexity of the model on the given text. Given this loss function, we will proceed to investigate the loss landscape of our model.

2.5.2 Loss Landscape

Since the loss landscape of a language model is in n -dimension, where n is its number of parameters, it is difficult to be directly visualized. We will investigate the loss landscape of our TinyStories v2 model from Section 2.2 by perturbing its weights and computing the perplexity of the perturbed models.

Straight Line

First, we will sample models along a simple line in the following way.

$$\theta' = \theta + \epsilon, \epsilon \in \{-0.0020 \dots 0.0015\} \quad (2.21)$$

Note that the blue line in the figure represents $\epsilon \in \{-0.0015 \dots 0.0015\}$. From the plot in Figure 2.4, we notice that there is a “basin” of low loss, and that moving towards the negative direction caused less degradation than moving towards the positive direction.

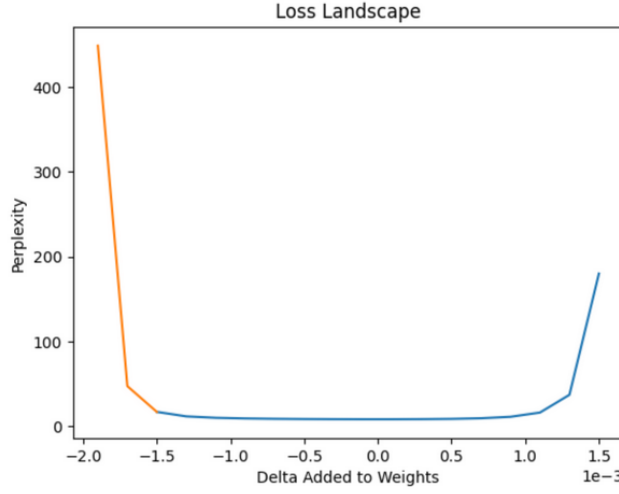


Fig. 2.4: Models Along a Line

Moving Randomly

Next, we sample models stochastically from the following distribution. We sample 5 models for each σ .

$$\theta' \sim \mathcal{N}(\theta, \sigma^2 \mathbf{I}), \sigma^2 \in \{0.0005 \dots 0.0040\} \quad (2.22)$$

Note that we cap the perplexity at 100 in Figure 2.5 for better visualization, because some perplexities are extremely large.

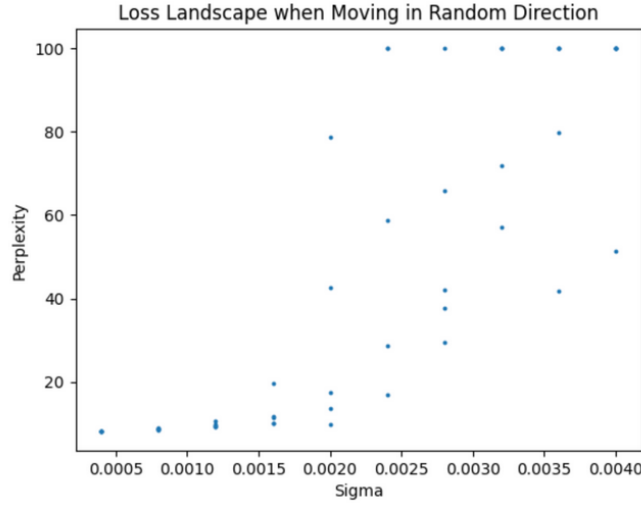
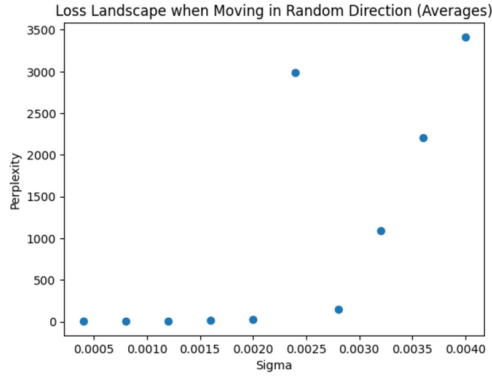
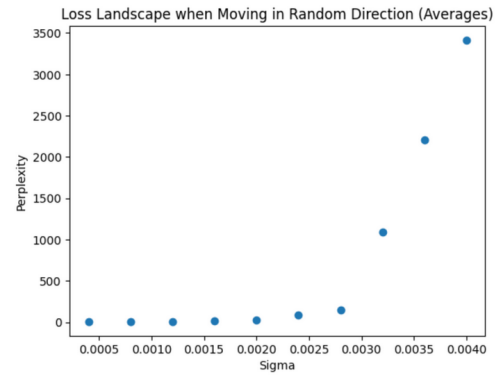


Fig. 2.5: Models from Gaussian Distribution

In Figure 2.6a, we plot the average perplexities of each σ . We notice an outlier at $\sigma = 0.0024$, and removing it yields Figure 2.6b.



(a) Averages



(b) Averages (Outlier Removed)

Fig. 2.6: Loss Landscape of Models from Gaussian Distribution

This further confirms the observation that when perturbing a model, the direction of perturbation is important, and some directions yield terrible results (like the outlier) while some directions preserve the model's accuracy more.

Towards Another Model

We pretrain another model with the same configuration as the base model, and move from the base model to the other model. The distance between these models, $\|\theta - \theta'\|_2$, is 75.6, which means they're quite far apart, and the magnitude of perturbation is much greater than the previous experiments.

Note that when the x-axis is 0, the model is exactly the base model, and when the x-axis is 1, the model is exactly the other pretrained model.

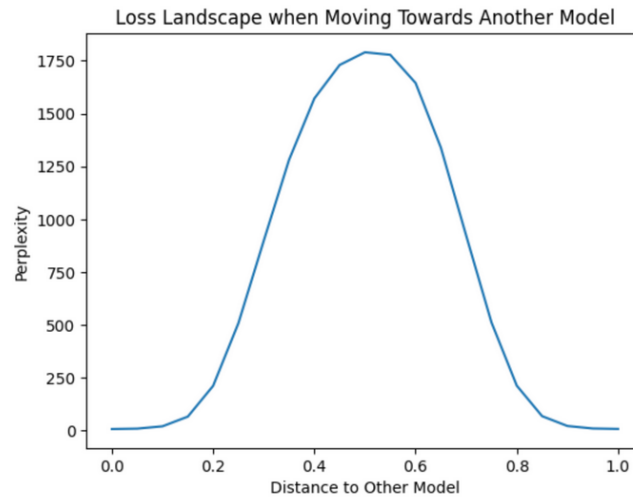


Fig. 2.7: Moving Towards Another Model

We observe from Figure 2.7 that even though the magnitude of perturbation is much greater than the previous experiments, the increase in loss is not as significant. This shows that the model is less sensitive to perturbations in this direction.

2.5.3 Discussion

The key insight from this experiment is the fact that

Consider adding an experiment: loss when moving towards the direction of the gradient

Write how some directions are better than others, and worst directions cause significant degradation

Chapter 3

Literature Review

3.1 AdaRound

3.1.1 Notation

Let us establish the notation for variables in Table 3.1.

Table 3.1: Notation for this Literature Review

Symbol	Description
w_j	Weights of a layer j
x_j	Input to a layer j
b	Target bit-width quantization
scale	Continuous scaling factor
$\lfloor \cdot \rfloor$	Round-to-nearest-integer operator
$\lfloor \cdot \rfloor$	Floor operator
$\mathbf{W}, \hat{\mathbf{W}}$	Matrices of unquantized and quantized weights
$\mathcal{L}$	A loss function

3.1.2 Introduction to AdaRound

AdaRound [12] is a novel rounding method, based on the key insight that round-to-nearest is suboptimal for post-training quantization (PTQ).

The authors' proposed method is to instead minimize the difference in task loss of the quantized model and the unquantized model. The task loss of layer j of the unquantized model is $\mathcal{L}(x, y, w_j)$, and the task loss of the same layer of the quantized model $\mathcal{L}(x, y, w_j + \Delta w_j)$. Therefore, the author's objective is the following.

$$\arg \min_{\Delta w_j} (\mathcal{L}(x, y, w_j + \Delta w_j) - \mathcal{L}(x, y, w_j)) \quad (3.1)$$

The author use a second order Taylor expansion to approximate the above objective.

$$\mathcal{L}(x, y, w_j + \Delta w_j) - \mathcal{L}(x, y, w_j) \approx \nabla \mathcal{L}(x, y, w_j)^T \Delta w_j + \frac{1}{2} \Delta w_j^T \mathbf{H} \Delta w_j \quad (3.2)$$

Where $\mathbf{H}$ is the Hessian $\nabla^2 \mathcal{L}(x, y, w_j)$. The authors then ignore the first term by asserting that the gradient $\nabla \mathcal{L}(x, y, w_j)$ is 0 for converged pretrained models, which simplifies into the following.

$$\mathcal{L}(x, y, w_j + \Delta w_j) - \mathcal{L}(x, y, w_j) \approx \frac{1}{2} \Delta w_j^T \mathbf{H} \Delta w_j \quad (3.3)$$

Finally, the author formulates the optimization of weigh rounding for layer j as the following Quadratic Unconstrained Binary Optimization (QUBO) problem.

$$\arg \min_{\Delta w_j} \Delta w_j^T \mathbf{H} \Delta w_j \quad (3.4)$$

The above problem is NP-hard. To solve this efficiently, AdaRound uses the local mean squared error alongside a regularizer as a proxy. The regularizer encourages the optimization variables to converge towards either 0 or 1.

Derivation

$$\arg \min_{\mathbf{V}} \|\mathbf{W}x - \hat{\mathbf{W}}x\|_F^2 + f_{\text{reg}}(\mathbf{V}) \quad (3.5)$$

The regularizer uses an annealed parameter β and is defined as follows.

$$f_{\text{reg}}(\mathbf{V}) = \sum_{i,j} 1 - |2h(\mathbf{V}_{i,j}) - 1|^\beta \quad (3.6)$$

The above objective is then optimized over

$$\hat{\mathbf{W}} = \text{scale} \cdot \text{clamp} \left(\left\lfloor \frac{\mathbf{W}}{\text{scale}} \right\rfloor + h(\mathbf{V}), q_{\min}, q_{\max} \right), \quad (3.7)$$

$$q_{\min} = -2^{b-1}, \quad (3.8)$$

$$q_{\max} = 2^{b-1} - 1. \quad (3.9)$$

The authors define h as a rectified sigmoid as follows, although it can be any differentiable function with a range of $h(\mathbf{V}_{i,j}) \in [0, 1]$.

$$h(\mathbf{V}_{i,j}) = \text{clamp}(\sigma(\mathbf{V}_{i,j})(\zeta - \gamma) + \gamma, 0, 1), \quad (3.10)$$

$$\zeta = 1.1, \quad (3.11)$$

$$\gamma = -0.1. \quad (3.12)$$

The use of a rectified sigmoid is justified as it has non-vanishing gradients near 0 and 1, and helps the learning process as they previously used a regularizer to encourage $h(\mathbf{V}_{i,j})$ to move to either 0 or 1.

3.1.3 AdaRound Algorithm

We implement AdaRound in Algorithm 3.

Algorithm 3 AdaRound

Require: $\mathbf{W}$, X (calibration data), t (number of iterations), b (target bit width), β (regularization parameter)

Ensure: Quantized model $\mathbf{W}_Q$, scales

```

1:  $x_{in} \leftarrow \text{CACHEACTIVATIONS}(\mathbf{W}, X)$   $\triangleright$  Forward pass on calibration data to cache
   input activations
2: for  $w_j$  in  $\mathbf{W}$  do
3:    $q_{\min} \leftarrow -2^{b-1}$ 
4:    $q_{\max} \leftarrow 2^{b-1} - 1$ 
5:    $\text{scales}_j \leftarrow \frac{\max(w_j)}{q_{\max}}$   $\triangleright$  Store scale for output
6:    $\mathbf{V}_j \leftarrow h^{-1}(w_j - \lfloor w_j \rfloor)$   $\triangleright$  Initialize  $\mathbf{V}_j$  so that it corresponds to the fraction of
   each weight
7:   for  $i \in \{0 \dots t\}$  do
8:      $\beta \leftarrow \text{ANNEAL}(\beta)$ 
9:      $\hat{w}_j \leftarrow \text{clamp}\left(\lfloor \frac{w_j}{\text{scales}_j} \rfloor + h(\mathbf{V}_j), q_{\min}, q_{\max}\right)$ 
10:     $\mathcal{L} \leftarrow \|w_j x_{in} - \hat{w}_j x_{in}\|_F^2 + \sum_{i,j} (1 - |2h(\mathbf{V}_{i,j}) - 1|^\beta)$ 
11:     $\mathbf{V} \leftarrow \mathbf{V} - \nabla \mathcal{L}$   $\triangleright$  via autograd
12:   end for
13:    $\mathbf{W}_{Q_{i,j}} \leftarrow \begin{cases} \lceil w_{i,j} \rceil, & \text{where } h(\mathbf{V}_{i,j}) \geq 0.5 \\ \lfloor w_{i,j} \rfloor, & \text{otherwise} \end{cases}$   $\triangleright$  Finalize quantization
14: end for
15: return  $\mathbf{W}_Q$ , scales

```

Report
AdaRound
results

3.1.4 Empirical Results

3.2 Inversion Method

Chapter 4

Proposed Method

4.1 AdaRound with Gradient Awareness

To reconcile AdaRound’s assumption that the gradient term in ?? can be ignored, and Central Flows’ evidence that the gradient is nonzero even for pretrained converged models, we propose a modified version of AdaRound to incorporate awareness of the local gradient.

In order to minimize

$$\mathbb{E} \left[g^{(w)^T} \Delta W \right], \quad (4.1)$$

we can attempt to force ΔW to have the same sign as $-g^{(w)^T}$. We approximate this by finetuning a model on a small amount of training data, and compute the Exponential Moving Average (EMA) of the model during each iteration. We then use $\text{sign}(W'_{\text{avg}} - W)$, the sign of the difference between the weights before and after finetuning, as a proxy to $-\text{sign}(g^{(w)^T})$. Finally, we add the Binary Cross Entropy loss between the current rounding directions and $\text{sign}(W'_{\text{avg}} - W)$ as a penalty term to AdaRound’s optimization objective.

Algorithm 4 Proposed Method (Finetuning)

Require: W , X (calibration data), η (learning rate), B (batch size)

Ensure: Finetuned model W' , EMA of models during finetuning W'_{avg}

```

1:  $W' \leftarrow W$ 
2:  $W'_{\text{avg}} \leftarrow W$ 
3:  $N \leftarrow \frac{X}{B}$  ▷ Number of Iterations
4:  $\alpha \leftarrow \frac{2}{N+1}$  ▷ EMA Smoothing Factor
5: for each batch  $x_b$  in  $X$  do
6:    $\hat{x}_b \leftarrow W'(x_b)$  ▷ Forward pass
7:    $\mathcal{L} \leftarrow \text{CrossEntropyLoss}(\hat{x}_b, x_b)$ 
8:    $W' \leftarrow W' - \eta \cdot \nabla \mathcal{L}$  ▷ Backward pass
9:    $W'_{\text{avg}} \leftarrow \alpha \cdot W'_{\text{avg}} + (1 - \alpha) \cdot W'$ 
10: end for
11: return  $W'$ ,  $W'_{\text{avg}}$ 

```

Algorithm 5 Proposed Method (Calibration)

Require: W, W'_{avg}, X (calibration data), η (learning rate), t (number of iterations)**Ensure:** Quantized model W_q

1: (TODO)

2: **return** W_q

Chapter 5

Evaluation

Chapter 6

Conclusion

Bibliography

- [1] C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, Vol. 27, No. 3, pp. 379–423, 1948.
- [2] B. Ebanks, P.K. Sahoo, and W. Sander. *Characterizations of Information Measures*. World Scientific Publishing Company, 1998.
- [3] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report, May 2025.
- [4] Samuel L. Smith, Benoit Dherin, David G. T. Barrett, and Soham De. On the Origin of Implicit Regularization in Stochastic Gradient Descent, 2021.
- [5] Ronen Eldan and Yuanzhi Li. TinyStories: How Small Can Language Models Be and Still Speak Coherent English?, May 2023.
- [6] Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. GPT-Neo: Large scale autoregressive language modeling with meshtensorflow. Zenodo, October 2021.
- [7] Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing, August 2018.
- [8] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models, February 2023.
- [9] F. Jelinek, R. L. Mercer, L. R. Bahl, and J. K. Baker. Perplexity—a measure of the difficulty of speech recognition tasks. *The Journal of the Acoustical Society of America*, Vol. 62, No. S1, pp. S63–S63, December 1977.
- [10] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A White Paper on Neural Network Quantization, June 2021.
- [11] S. Kullback and R. A. Leibler. On Information and Sufficiency. *The Annals of Mathematical Statistics*, Vol. 22, No. 1, pp. 79–86, March 1951.
- [12] Markus Nagel, Rana Ali Amjad, Mart van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or Down? Adaptive Rounding for Post-Training Quantization, June 2020.

A

Appendix